

Introduction to Supervised Learning

Applied Machine Learning — Session 1, Chapter 3

Supervised Learning

Learning from labeled examples to predict new cases.

- X = features (what we know) — e.g. petal length & width
- y = label / target (what we want to predict) — e.g. species
- $\hat{y}$ = the model's prediction for a new X

Task	Target	Example
Regression	Continuous number	House price: €285 000
Classification	Discrete category	Spam / not spam • Iris species

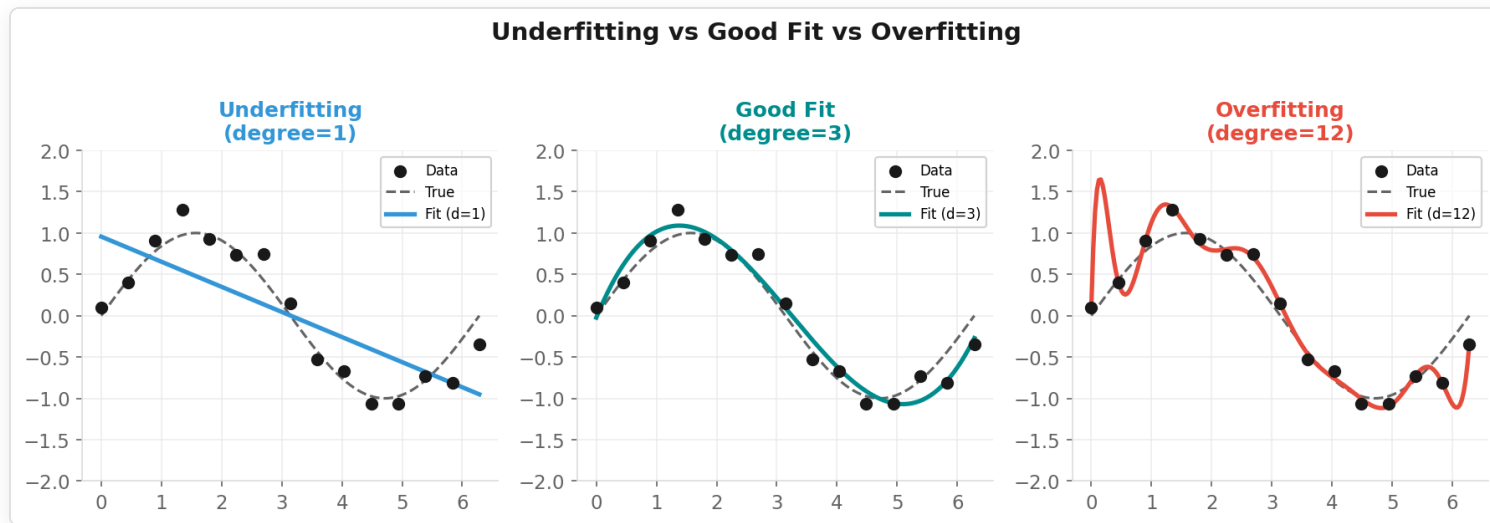
The Protocol: Train, Then Test on Unseen Data

- | | |
|------------|--|
| ① Split | train test (test locked away – Ch02) |
| ② Baseline | "always predict the majority class" – the bar to beat |
| ③ Train | model.fit(X_train, y_train) |
| ④ Evaluate | model.score(X_test, y_test) ← the number we care about |

Training accuracy = how well the model repeats what it has seen

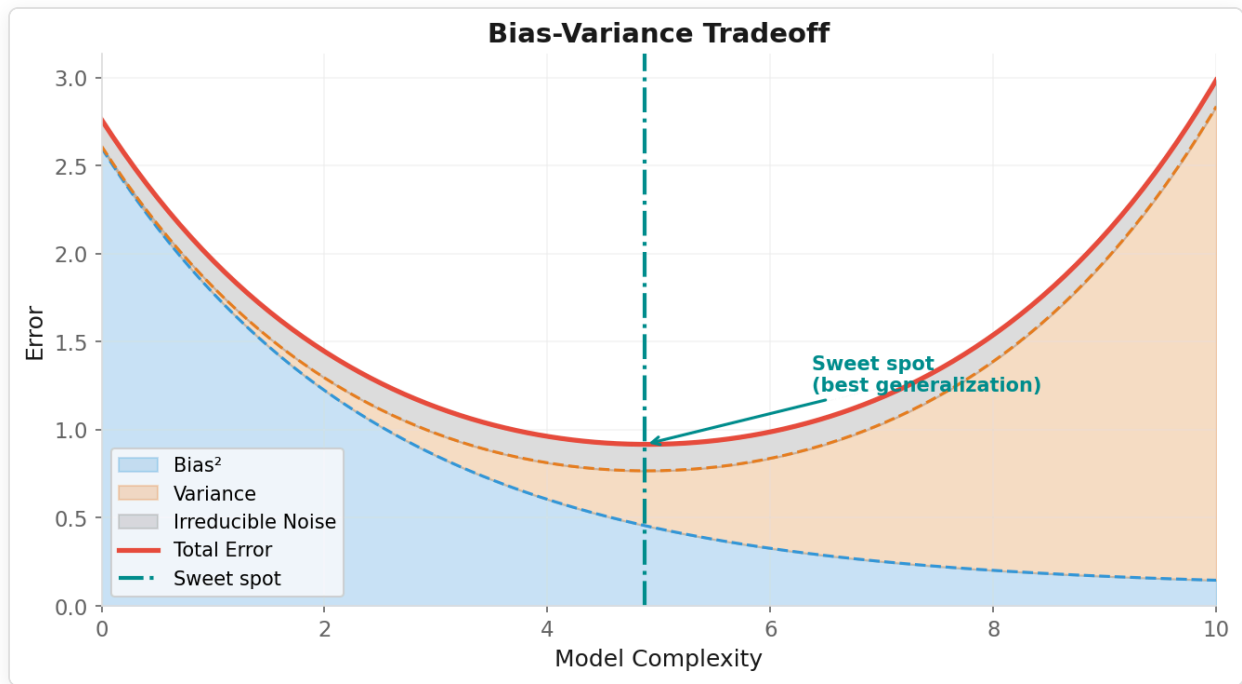
Test accuracy = how well it **generalises** to new data ← what matters

Underfitting vs Overfitting



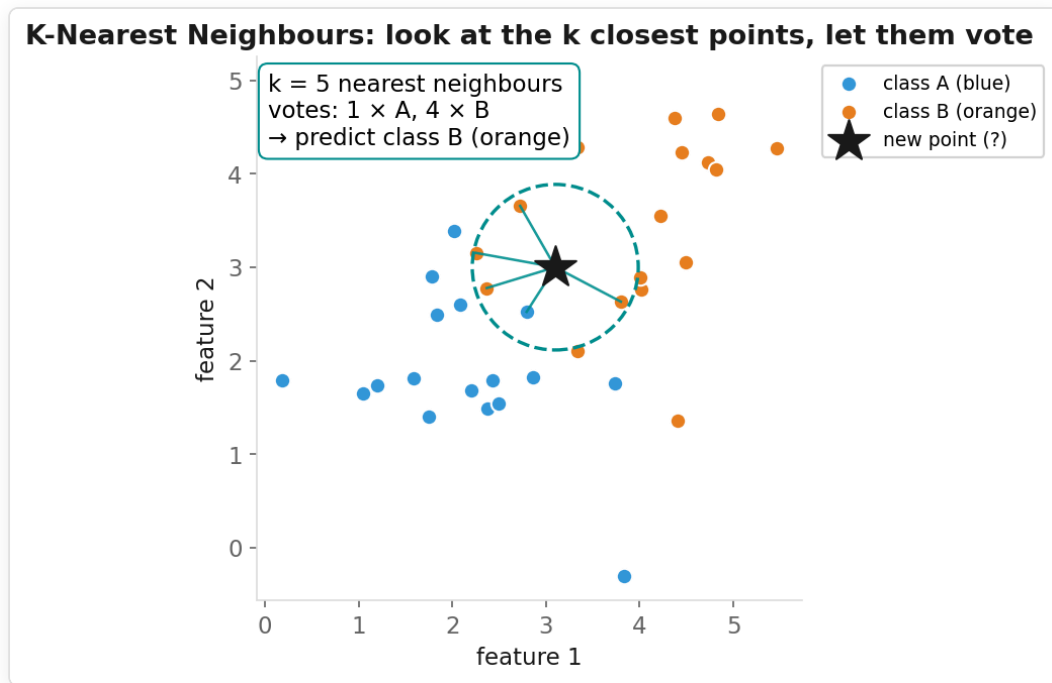
- **Underfitting:** model too simple $\rightarrow$ train and test error high (high bias)
- **Overfitting:** model memorises noise $\rightarrow$ train error low, test error high (high variance)

The Bias–Variance Tradeoff



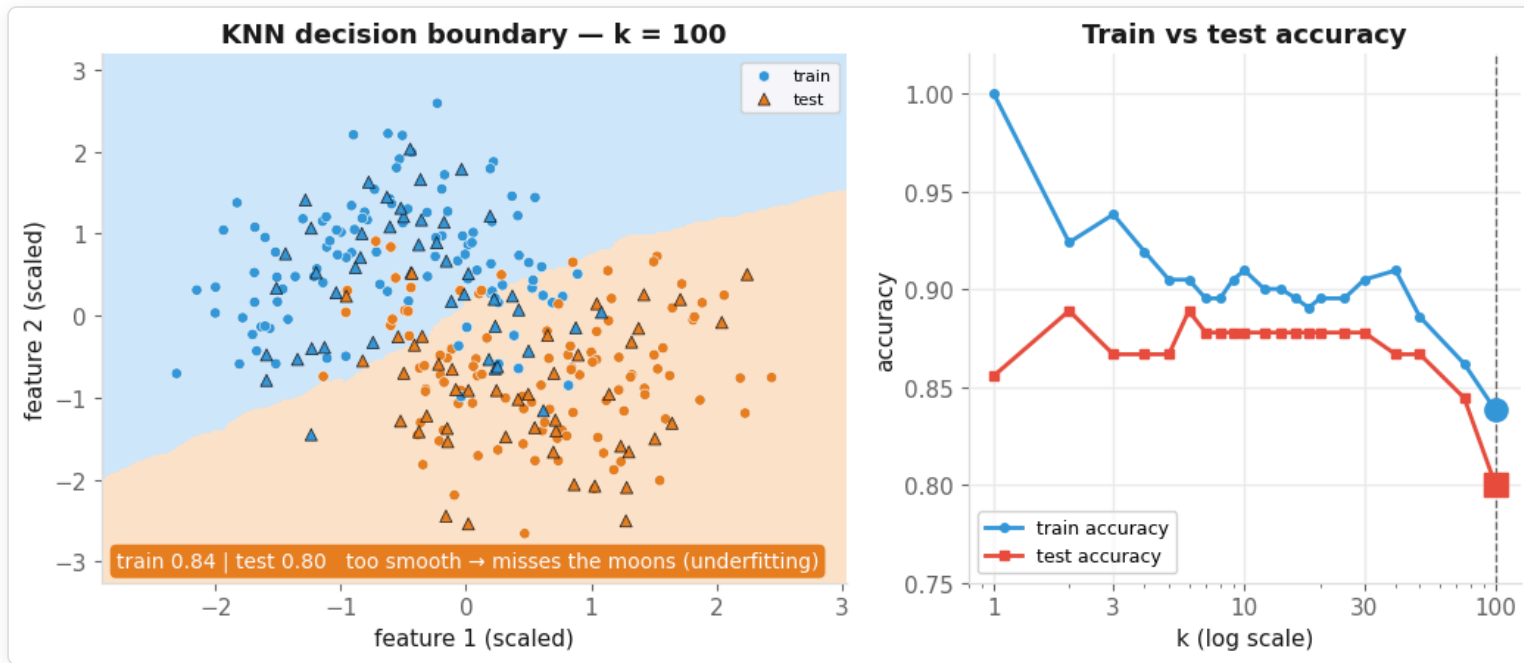
Goal: the sweet spot where test error is minimal — neither too simple nor too flexible.

K-Nearest Neighbours (KNN)

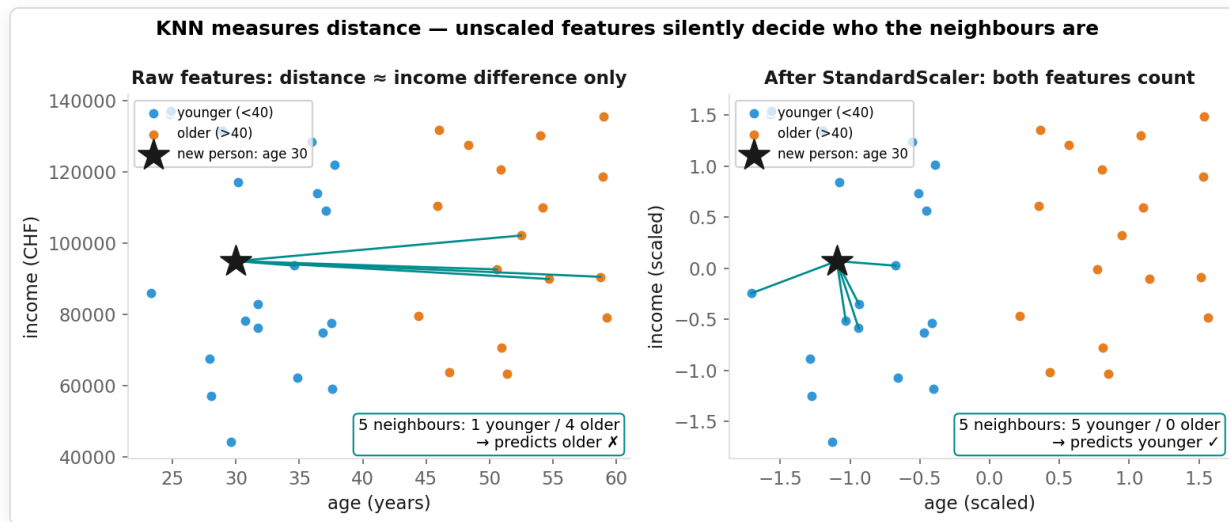


No formula, no training — to predict a new point: ① measure the distance to all training points, ② take the k closest, ③ majority vote.

The Effect of k



KNN Needs Scaled Features



Distances are dominated by the feature with the largest range → `StandardScaler` before KNN — inside the pipeline.

Hyperparameter vs Parameter

Hyperparameter

Chosen by YOU before training

KNN: k (number of neighbours)

Polynomial: degree

Tree: max depth

Tuned with validation data / CV

Parameter

Learned by the ALGORITHM from data

Linear regression: slope, intercept

Logistic regression: weights

(KNN has none — it stores the data)

The sklearn API — One Interface for All Models

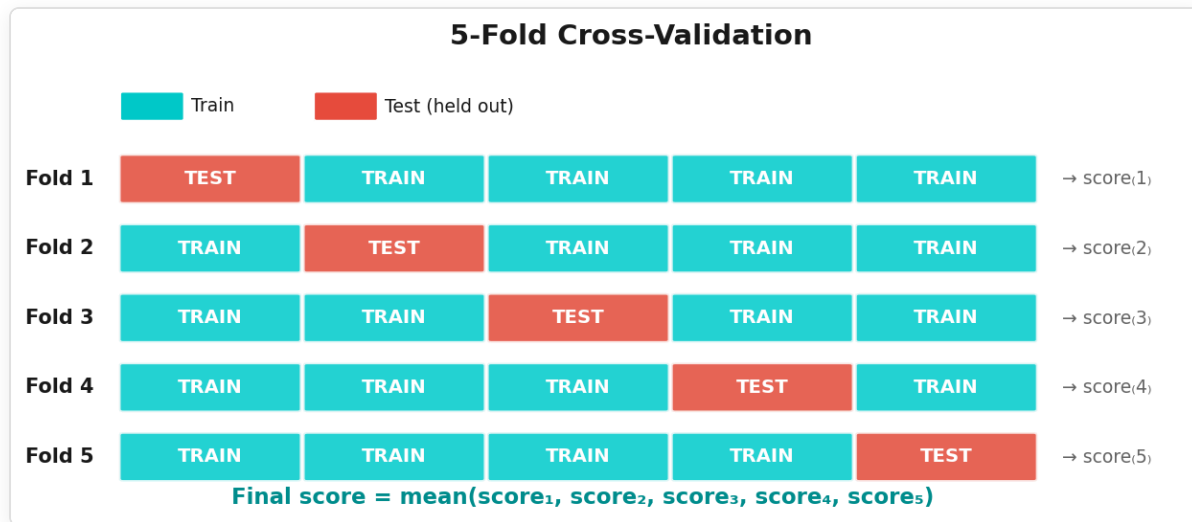
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

model = Pipeline([('scaler', StandardScaler()),          # Ch02 preprocessing ...
                  ('knn', KNeighborsClassifier(n_neighbors=5))]) # ... + a model at the end

model.fit(X_train, y_train)          # 1. train (scaler AND knn fit on train only)
y_pred = model.predict(X_test)      # 2. predict
model.score(X_test, y_test)         # 3. evaluate (accuracy)
```

Every sklearn model works exactly this way — swap `KNeighborsClassifier` for any other estimator. ✓

Preview: Cross-Validation (Ch04)



One split = one noisy number. **k-fold CV** repeats the split k times and averages → more reliable estimate, used to *choose* hyperparameters. **We will use it in Ch04.**

Quick Check

A KNN model reaches **100 % training accuracy** and **72 % test accuracy**. Baseline is 65 %.

1. Is this over- or underfitting?
2. Should you **increase** or **decrease** k ?
3. Your colleague suggests reporting the 100 % — what do you say?

Quick Check

A KNN model reaches **100 % training accuracy** and **72 % test accuracy**. Baseline is 65 %.

1. Is this over- or underfitting?
2. Should you **increase** or **decrease** k?
3. Your colleague suggests reporting the 100 % — what do you say?

Answers: 1 → Overfitting (train » test). 2 → **Increase** k → smoother boundary, less variance. 3. Training accuracy is not evidence of anything — report test accuracy, and mention it is only 7 points above the baseline.

Now: Live Demo, Then Exercises

Demo (~10 min): `02-examples/ch03_supervised_intro_examples.ipynb` two moons → baseline →

`Pipeline(scaler, KNN)` fit/predict/score → boundaries for $k = 1 / 7 / 51$ → k sweep (train vs test) → Penguins with/without scaler → CV taste

Exercises (~10 min): `03-exercises/ch03_supervised_intro_exercises.ipynb` Task 1 split → Task 2 baseline + KNN

→ Task 3 k sweep → Task 4 plot & interpret • Bonus: Penguins pipeline with `SimpleImputer` (Ch02 skills), scaler on/off, CV taste

Key Takeaways

- Supervised learning = learn a mapping $X \rightarrow y$ from labelled examples
- Protocol: split $\rightarrow$ **baseline** $\rightarrow$ fit on train $\rightarrow$ score on **test**
- Underfitting (too simple) vs. overfitting (memorises noise) – judged by the train/test gap
- KNN: distance + majority vote; **k** is a hyperparameter; **scale first**
- sklearn: `Pipeline(...).fit / .predict / .score` – same for every model
- Cross-validation $\rightarrow$ Ch04

Bonus — Further Reading

- **Loss functions** (what `fit` actually minimises): MSE for regression, cross-entropy for classification → Ch04 / Ch05
- **Distance metrics** for KNN: Euclidean (default), Manhattan, cosine —
`KNeighborsClassifier(metric=...)`
- **Weighted KNN:** `weights='distance'` lets closer neighbours count more
- **KNN for regression:** `KNeighborsRegressor` averages the neighbours' values

Next: Chapter 4

Regression Models

"Time to meet the algorithms. Starting with the classics."